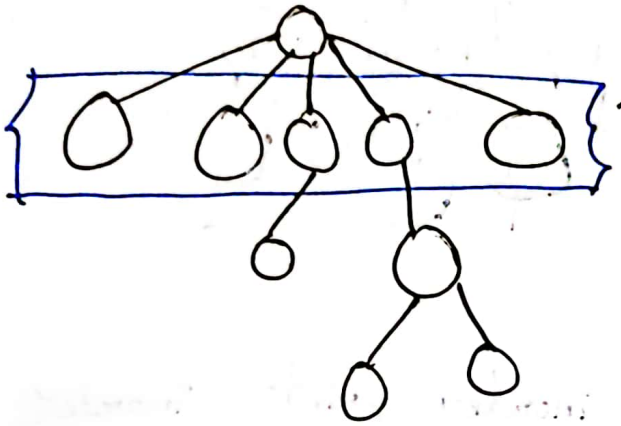
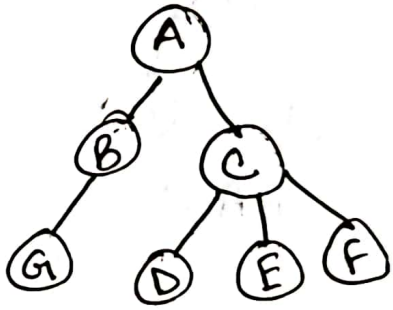
14-08-23

① Create a node class for a tree whose degree is dynamic

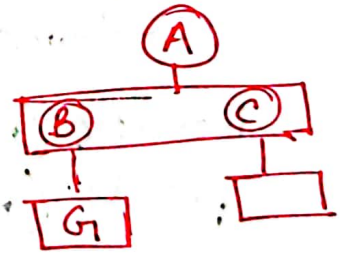


Here, degree = 3.



Class Node:

```
def __init__ (self, elem):
    self.elem = elem
    self.children = [ ] # a list of nodes.
```



#driver code

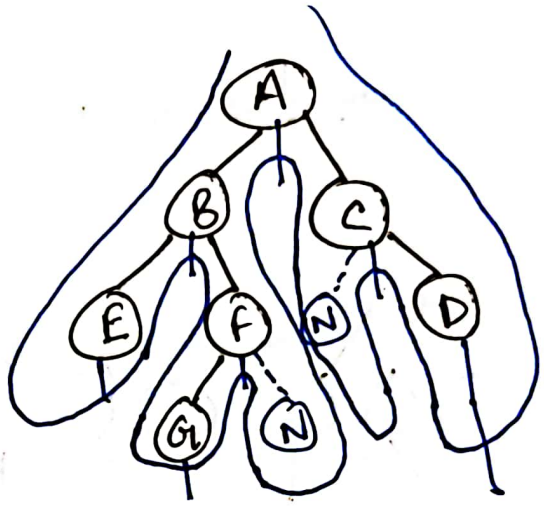
```
root = Node ('A')
n1 = Node ('B')
root.children.append(n1)
n2 = Node ('C')
root.children.append(n2)
n3 = Node ('G')
n1.children.append(n3)
```

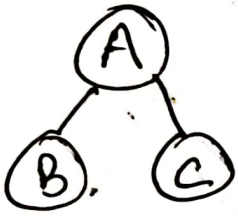
Healthcare
Emistat®
Ondansetron
CamScanner

Traversal

- inorder

EBGFACD





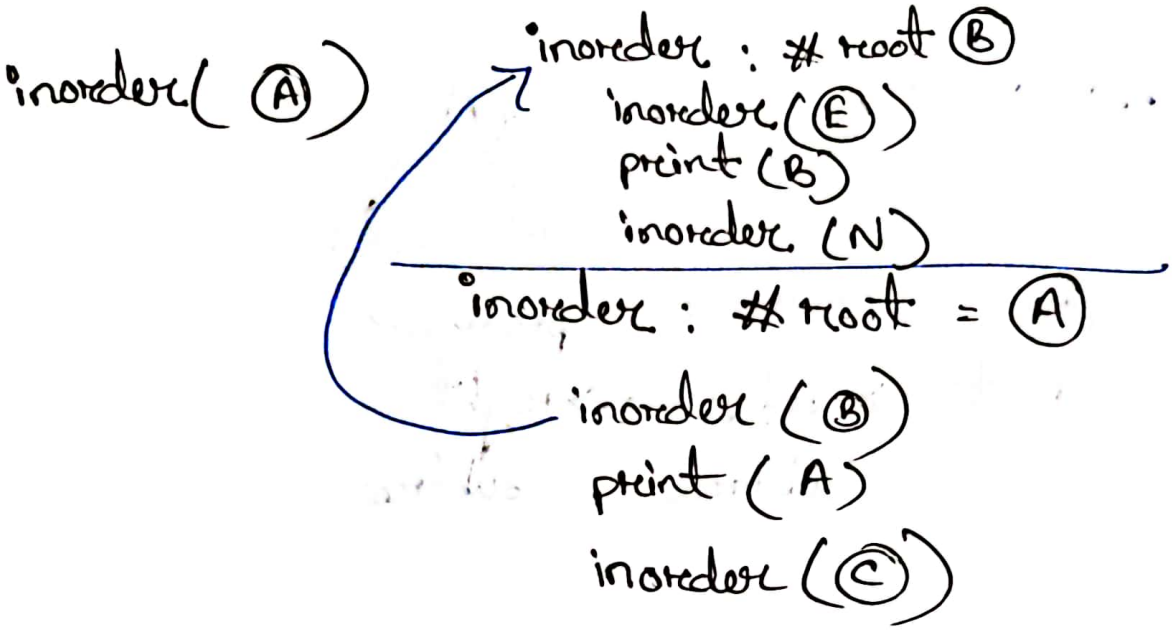
BAC

inorder print inorder
left – root – right
subtree subtree

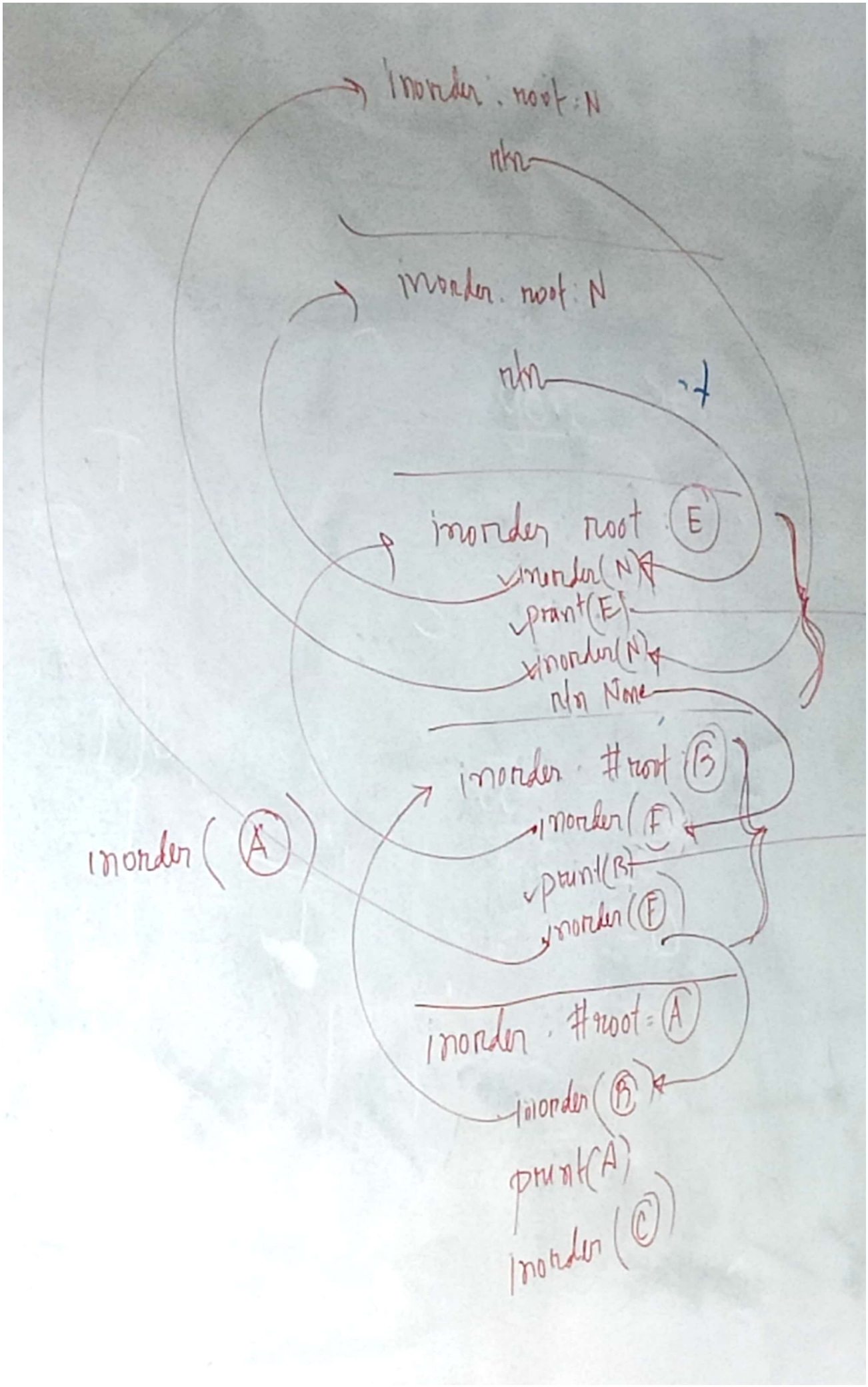
Code

```
def inorder (root):
    if root == None:
        return
    inorder (root.left)
    print (root.elem)
    inorder (root.right)
```

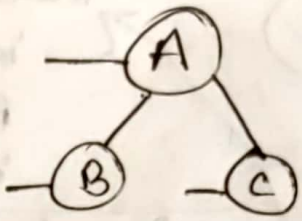
CamScanner



Healthcare
Emistat®
Ondansetron
CamScanner


inorder . root : N
rtn
inorder . root : N
rtn
inorder root (E)
inorder(N)
print(E)
inorder(N)
rtn None
inorder # root (B)
inorder(F)
print(B)
inorder(E)
inorder(A)
inorder # root = (A)
inorder(B)
print(A)
inorder(C)

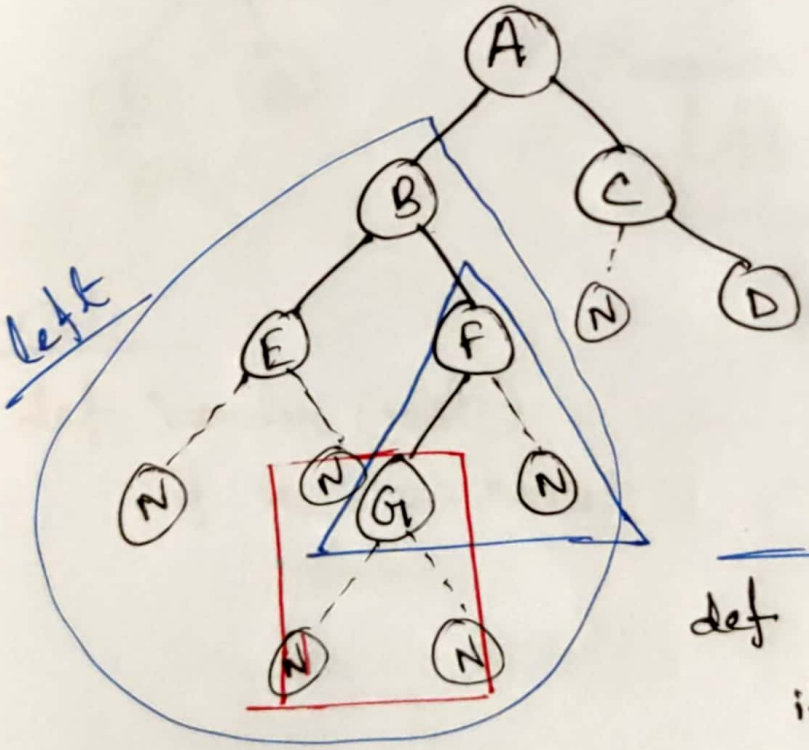
• preorder



print preorder preorder

root → left → right

subtree subtree



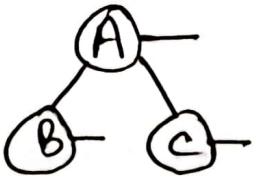
left result

A B E F G

Code

```
def preorder (root):
    if root == None:
        return
    print (root.elem)
    preorder (root.left)
    preorder (root.right)
```

• Postorder



left → right → root

Code ⇒ ↳ see previous one

Healthcare

Emistat®
Ondansetron

CamScanner

count the num of nodes in a tree.

of nodes in a tree = (# of nodes in a tree rooted at left subtree) + (# of nodes in a tree rooted at right subtree) + 1
↳ for root.

count_node (A) = count_node (B) + count_node (C) + 1.

```
def count_node (root):
    if root == None:
        return 0.
```

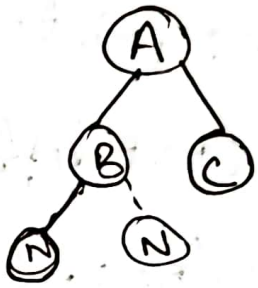
```
left_subtree.count = count_node (root.left)
right_subtree.count = count_node (root.right)
total = left_subtree_count + right.subtree_count
        + 1
return total.
```

CamScanner

print the leaf nodes in a Binary tree :

=> ~~def leaf_print (root):~~

~~if root == None:~~
~~return~~
~~A.root.~~
~~node.root.~~




```
def leaf_print (root):
    if root.left == None and root.right == None:
        print (root.elem)
        return
    if root == None:
        return
    leaf_print (root.left)
    leaf_print (root.right)
```

Base case

Base case.

Healthcare
Emistat® Ondansetron
CamScanner

(##) print the nodes with one child in a Binary tree:

```
def leaf_print(root):
    if (root.left != None and root.right == None)
    or (root.right != None and root.left == None):
        print(root.elem)
        return.
```

CamScanner